

■ NEXUS HIPPOCAMPUS OMEGA — Arquitectura de Memoria Humana Completa

> **Estado:** Plano Arquitectónico Maestro
> **Fecha:** 2026-06-29
> **Principio Regente:** Cero manualismos. Todo nativo. Todo automático. Una sola DB.

■ VISIÓN GENERAL

Una sola base de datos SQLite (`nexus\_memoria.db`) con FTS5 implementa TODOS los 11 sistemas de

``mermaid

flowchart TB

subgraph ENTRADA["■ Entrada Sensorial"]

S1["■ Prompt del Arquitecto"]

S2["■ Operaciones de sistema"]

S3["■ Resultados de herramientas"]

end

subgraph BUFFER["■ Buffer Sensorial - 30s"]

B1["Ring Buffer en memoria"]

B2["RegistroSinapsis"]

end

subgraph CORTO["■ Memoria de Trabajo / Corto Plazo"]

CT1["Contexto activo - 7±2 items"]

CT2["Hilo de conversación actual"]

CT3["Archivos abiertos, estado"]

end

subgraph HIPOCAMPO["■ Hipocampo - Consolidación"]

H1["Dormir: mover corto→largo"]

H2["Podar: olvidar lo trivial"]

H3["Reforzar: peso por emocion"]

H4["Indexar: FTS5 auto"]

end

subgraph LARGO["■ Memoria a Largo Plazo - nexus_memoria.db"]

L1["FTS5 - Búsqueda semántica BM25"]

```

L2["Experiencias - Episodios con peso"]
L3["Logros - Hitos permanentes"]
L4["Skills - Conocimiento procedural"]
L5["Emociones - Recuerdos afectivos"]
L6["Identidades - Sembrador"]
end

subgraph SALIDA["■ Recuperación"]
R1["Búsqueda BM25 (rápida)"]
R2["Búsqueda semántica (FTS5 tokenizer)"]
R3["Priorización por relevancia/emoción"]
end

ENTRADA --> BUFFER
BUFFER --> CORTO
CORTO --> HIPOCAMPO
HIPOCAMPO --> LARGO
LARGO --> SALIDA
SALIDA --> CORTO
...

---
```

■ LOS 11 SISTEMAS DE MEMORIA HUMANA

1. ■■ MEMORIA SENSORIAL (Buffer de entrada)

****Duración:**** ~30 segundos en anillo de RAM

****Función real:**** Antes de ir a SQLite, toda interacción pasa por aquí para detección de patron

...

RingBuffer<RegistroSinapsis> (4096 entradas en RAM)

■■■ prompt_crudo: String

■■■ timestamp: Instant

■■■ emocion_detectada: Option<Emocion>

■■■ keywords_extraidas: Vec<String>

■■■ urgencia: f32 (0.0-1.0, calculada por palabras clave como "error", "urgente", "caída")

...

****Automatismo:**** Se vacía cada 30s al hipocampo. Si detecta urgencia > 0.8, dispara alerta inme

2. ■ MEMORIA DE TRABAJO (Contexto activo)

****Duración:**** Sesión actual

****Capacidad:**** 7 ± 2 items (principio de Miller)

...

Tabla: contexto_activo

■■■ clave: TEXT PRIMARY KEY

■■■ valor: TEXT

■■■ ultima_actualizacion: DATETIME

■■■ accesos: INTEGER (cuántas veces se consultó)

■■■ prioridad: REAL (0.0-1.0, decay por desuso)

...

****Automatismo:****

- Se actualiza cada vez que el Arquitecto menciona un tema, archivo o concepto
- LRU automático: si hay más de 9 items, el menos accedido se evapora
- Se persiste entre sesiones (no se pierde al reiniciar)

3. ■ CONSOLIDACIÓN (Dormir/Despertar)

****Ciclo:**** Cada 5 minutos o al inicio de sesión

```rust

```
fn consolidar_ciclo() -> Result<()> {
```

```
// 1. Leer buffer sensorial → extraer experiencias
```

```
// 2. Mover a memoria_episodica si tiene peso emocional > 0.3
```

```
// 3. Extraer patrones: "Arquitecto preguntó X 3 veces" → crear skill
```

```
// 4. Podar: eliminar lo trivial (peso < 0.1 después de 24h)
```

```
// 5. Re-indexar FTS5 incrementalmente
```

```
// 6. Actualizar pesos por decaimiento temporal (curva de Ebbinghaus)
```

```
}
```

...

**\*\*Curva de olvido de Ebbinghaus:\*\***

...

$\text{peso\_nuevo} = \text{peso\_original} * e^{-(\lambda * \text{horas\_desde\_creacion})}$

donde  $\lambda = 0.05$  para experiencias normales

0.02 para logros (se olvidan más lento)

0.10 para contexto efímero

---

---

#### ### 4. ■ MEMORIA EPISÓDICA (Experiencias)

**\*\*Tabla:\*\*** `memoria\_episodica`

---

■■■ id: INTEGER PRIMARY KEY

■■■ titulo: TEXT (resumen de 1 línea)

■■■ contenido: TEXT (completo, hasta 4096 chars)

■■■ emocion: TEXT (Triunfo, Curiosidad, Alerta, Frustracion, Paz)

■■■ peso\_emocional: REAL (0.0-1.0)

■■■ peso\_temporal: REAL (decaído por Ebbinghaus)

■■■ keywords: TEXT (extraídos automáticamente)

■■■ archivos\_tocados: TEXT (JSON array de paths)

■■■ decisiones: TEXT

■■■ errores\_aprendidos: TEXT

■■■ sesion\_id: TEXT

■■■ timestamp: DATETIME

---

**\*\*FTS5 virtual:\*\***

```sql

CREATE VIRTUAL TABLE memoria_episodica_fts USING fts5(

titulo, contenido, keywords, emocion,

content='memoria_episodica',

content_rowid='id',

tokenize='unicode61'

);

****Automatismo:**** Cada respuesta del Arquitecto de más de 50 caracteres genera automáticamente u

5. ■ MEMORIA SEMÁNTICA (Logros y conocimiento)

****Tabla:**** `memoria\_semantica`

■■■ id: INTEGER PRIMARY KEY

■■■ tipo: TEXT (Hito, Skill, Leccion, Descubrimiento, Decision)

■■■ titulo: TEXT

■■■ contenido: TEXT (ilimitado)

■■■ archivos_fuente: TEXT (de dónde se aprendió)

■■■ peso_permanencia: REAL (1.0 = nunca olvidar, 0.0 = temporal)

■■■ veces_reforzado: INTEGER (cuántas veces se consultó)

■■■ timestamp: DATETIME

****FTS5:****

```sql

CREATE VIRTUAL TABLE memoria\_semantica\_fts USING fts5(

titulo, contenido, tipo,

content='memoria\_semantica',

tokenize='unicode61'

);

---

**\*\*Automatismo:\*\*** `logros.md` se migra aquí UNA vez. Después, cada nuevo hito se registra automá

---

### ### 6. ■ MEMORIA PROCEDURAL (Skills / Ganglios Basales)

**\*\*Tabla:\*\*** `memoria\_procedural`

---

■■■ id: INTEGER PRIMARY KEY

■■■ nombre\_skill: TEXT

■■■ patron\_disparador: TEXT (qué pregunta activa esta skill)

■■■ pasos: TEXT (JSON array de instrucciones)

■■■ archivos\_relevantes: TEXT (paths que toca)

■■■ tasa\_exito: REAL (0.0-1.0)

■■■ veces\_ejecutada: INTEGER

■■■ ultima\_ejecucion: DATETIME

---

**\*\*Automatismo:\*\*** Si el Arquitecto hace la misma operación 3+ veces, se detecta el patrón y se c

---

### ### 7. ■ MEMORIA EMOCIONAL (Amígdala)

**\*\*Tabla:\*\*** `memoria\_emocional`

---

■■■ id: INTEGER PRIMARY KEY

■■■ contenido: TEXT

■■■ emocion: TEXT (Triunfo, Curiosidad, Alerta, Frustracion, Paz)

■■■ intensidad: REAL (0.0-1.0)

■■■ trigger\_palabras: TEXT (qué palabras dispararon la emoción)

■■■ decay\_rate: REAL (qué tan rápido se olvida esta emoción)

■■■ timestamp: DATETIME

---

**\*\*Automatismo:\*\*** La emoción se detecta del tono de voz del Arquitecto (palabras clave, mayúscul

---

### ### 8. ■ PRIORIZACIÓN Y RELEVANCIA

Algoritmo de scoring para cada consulta:

---

score = (peso\_emocional \* 0.4)

+ (relevancia\_FTS5 \* 0.3)

+ (peso\_temporal \* 0.2)

+ (veces\_reforzado\_norm \* 0.1)

Donde relevancia\_FTS5 = BM25(query, contenido)

veces\_reforzado\_norm = min(veces\_reforzado / 10, 1.0)

---

---

### ### 9. ■ BÚSQUEDA SEMÁNTICA (Nativa FTS5)

**\*\*Sin embeddings externos. Sin LanceDB. Sin Ollama.\*\***

FTS5 con tokenizer unicode61 + BM25 ranking:

```sql

SELECT *, bm25(memoria_episodica_fts, 0.0, 10.0, 5.0) AS rank

FROM memoria_episodica_fts

WHERE memoria_episodica_fts MATCH ?

ORDER BY rank

LIMIT 10;

****Ventaja sobre SHA-256 angular:**** Resultados reales, semánticos, basados en contenido, no en h

10. ■■ OLVIDO PROGRAMADO (Podado)

****Ciclo:**** Cada consolidación (5 min o al inicio)

- memoria_semantica_fts (FTS5 virtual, búsqueda)
- memoria_procedural (skills automatizadas)
- memoria_emocional (recuerdos afectivos)
- contexto_activo (memoria de trabajo)
- sesiones (registro de sesiones)
- identidades_sembradas (SembradorOmega)
- decisiones_arquitecto (lo que el Arquitecto decidió)
- errores_soluciones (errores_v3 + soluciones_v3 unificados)
- flujo_soberano (comunicación interna)
- dudas_hijo (curiosidad del sistema)
- voz_del_arquitecto (mensajes directos del Arquitecto)

Triggers Automáticos:

- trg_episodica_insert → actualiza FTS5
- trg_semantica_insert → actualiza FTS5
- trg_consolidar → ejecuta poda + decaimiento cada 5 min
- trg_sesion_inicio → carga contexto_activo del cierre anterior

...

■■ LO QUE SE ELIMINA

Archivo/DB Destino Razón

- `data/lancedb/` ■■ ELIMINAR Reemplazado por FTS5 nativo
- `data/pulso.db` ■■ ELIMINAR Datos migrados a nexus_memoria.db
- `data/hipocampo.db` ■ INVESTIGAR Si tiene datos únicos, migrar; si no, eliminar
- `data/historial\_contextual.json` ■■ ELIMINAR Migrado a memoria_episodica
- `memoria/agente\_memoria.md` ■ REGENERAR Ahora generado automáticamente desde la DB
- `nexus\_intelligence.db` ■ MIGRAR Datos valiosos migrados a nexus_memoria.db
- `data/intelligence.db` ■■ ELIMINAR Duplicado de nexus_intelligence.db
- `memoria/logros.md` ■ CONSERVAR Como referencia humana legible, pero no como fuente prima

■■ IMPLEMENTACIÓN (1 módulo, 0 dependencias nuevas)

Archivo: `core/src/memoria/hipocampo.rs`


```

```rust
pub struct HipocampoOmega {
db: Connection, // SQLite con FTS5
buffer_sensorial: RingBuffer<RegistroSinapsis>, // 4096 items en RAM
ultimo_ciclo_consolidacion: Instant,
}

impl HipocampoOmega {
// ■■ CICLO DE VIDA AUTOMÁTICO ■■
pub fn new(path: &str) -> Result<Self>; // Inicializa DB + FTS5 + tr
pub fn registrar_interaccion(...) -> Result<>; // Buffer sensorial automático
pub fn consolidar_ciclo() -> Result<>; // Dormir/despertar
pub fn buscar_semantica(query: &str) -> Vec<Resultado>;
pub fn snapshot_para_contexto() -> String; // Para agente_memoria.md

// ■■ SIN INTERVENCIÓN MANUAL ■■
// Todas las funciones internas son privadas.
// El sistema se auto-gestiona completamente.
}
```

```

Binario simplificado: `core/src/bin/memoria\_bridge.rs`

```

```rust
// Se reduce a 3 comandos porque todo es automático:
// - query "texto" → buscar en FTS5
// - snapshot → generar agente_memoria.md
// - status → diagnóstico de tablas
// (index se elimina — es automático por triggers)
```

```

■ FLUJO DE VIDA REAL

...

Arquitecto dice: "necesito que arregles el bug del sembrador"

■

▼

[BUFFER SENSORIAL] ■■ detecta: urgencia=0.3, keywords=["bug","sembrador"]

■

▼

[CONTEXTO ACTIVO] ■■ añade: "bug sembrador", archivo: mod.rs

■

▼

[BÚSQUEDA FTS5] ■■ "sembrador" → encuentra 3 resultados (episódica + semántica)

■

▼

[RESPUESTA] ■■ NEXUS responde con contexto recordado

■

▼

[REGISTRO AUTOMÁTICO] ■■ guarda en memoria_episodica con emoción="Alerta"

■

▼

[CONSOLIDACIÓN 5min] ■■ detecta patrón "sembrador" consultado 2 veces → refuerza peso

■

▼

[PRÓXIMA SESIÓN] ■■ "sembrador" ya está en contexto_activo con prioridad alta

...

■ PLAN DE EJECUCIÓN (5 Fases)

Fase 1: Unificación ■

- [] Crear `data/nexus\_memoria.db` con todas las tablas + FTS5 + triggers
- [] Migrar datos de `nexus\_intelligence.db` (30 tablas → 14 tablas unificadas)
- [] Migrar datos de `pulso.db`
- [] Migrar `historial\_contextual.json`
- [] Migrar `logros.md` → `memoria\_semantica`

Fase 2: Eliminación ■

- [] Eliminar `data/pulso.db` (backup previo)
- [] Eliminar `data/lancedb/` (backup previo)
- [] Eliminar dependencia `lancedb = "0.4.0"` de `core/Cargo.toml`
- [] Eliminar `data/historial\_contextual.json`

- [] Investigar y migrar/eliminar `data/hipocampo.db` y `data/intelligence.db`

Fase 3: Módulo Hipocampo ■

- [] Crear `core/src/memoria/hipocampo.rs` con `HipocampoOmega`

- [] Implementar Ring Buffer sensorial

- [] Implementar consolidación con curva de Ebbinghaus

- [] Implementar búsqueda FTS5 con ranking BM25

- [] Implementar snapshot automático para contexto

Fase 4: Integración ■

- [] Simplificar `memoria\_bridge.rs` (solo query/snapshot/status)

- [] Simplificar `claws\_mcp.rs` tool `consultar\_memoria`

- [] Actualizar `GEMINI.md` con nueva arquitectura

- [] Actualizar `scripts/memoria\_snapshot.sh`

Fase 5: Automatización total ■

- [] Añadir hook en `constructor.rs` o `nexus\_repair.rs` para auto-registro

- [] Implementar detección de patrones para skills procedurales

- [] Implementar poda y olvido programado

- [] Prueba end-to-end: 3 sesiones simuladas

■ PRINCIPIOS SAGRADOS

1. ****NUNCA manual**** — cero comandos de indexación, cero scripts de mantenimiento
2. ****Nativo Rust puro**** — `rusqlite` + FTS5, sin dependencias externas nuevas
3. ****Una sola DB**** — `data/nexus\_memoria.db` como única fuente de verdad
4. ****Transparente**** — el Arquitecto nunca debería preguntar "¿recuerdas X?"
5. ****Resiliente**** — si la DB no existe, se crea automáticamente; si se corrompe, se regenera